

Question: Explain the Fibonacci Search Method for Univariate Optimization

Answer

The Fibonacci search method is a *derivative-free, sequential line-search* technique used to locate the minimizer of a *unimodal* function f over a closed interval $[a, b]$. A function is unimodal on $[a, b]$ if it has exactly one local minimum in that interval, decreasing up to the minimizer and increasing thereafter. Because the search relies only on comparing function values — never on gradients or derivatives — it is well suited to problems where f is expensive to evaluate, non-smooth, or not analytically differentiable.

The central idea is to maintain a *bracketing interval* that is guaranteed, at every stage, to contain the minimizer, and to shrink this interval as efficiently as possible — using the *fewest possible function evaluations* — until its width falls below a prescribed tolerance.

1. Role of the Fibonacci Numbers

The method takes its name from the Fibonacci sequence, defined by $F_1 = F_2 = 1$ and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 3$, giving 1, 1, 2, 3, 5, 8, 13, Before the search begins, enough Fibonacci numbers are generated so that the final term F_n first exceeds the ratio of the initial interval width to the desired tolerance, that is, $F_n > (b - a) / \text{tol}$. This value of n fixes, in advance, exactly how many iterations the search will require: a smaller tolerance demands a larger n and therefore more iterations, while a looser tolerance terminates sooner.

The Fibonacci numbers are not merely a stopping criterion; they determine the *precise placement* of the interior test points at every step, which is what allows the interval to shrink optimally.

2. Choosing the Initial Test Points

Rather than bisecting the interval as, for instance, a golden-section-style midpoint split might suggest, Fibonacci search places two interior points x_1 and x_2 inside $[a, b]$ using ratios of *consecutive* Fibonacci numbers:

$$x_1 = a + (F_{n-2} / F_n)(b - a), \quad x_2 = a + (F_{n-1} / F_n)(b - a)$$

Since $F_{n-2} < F_{n-1}$, it follows that $x_1 < x_2$, so x_1 always lies to the left of x_2 within the bracket. The objective function is then evaluated once at each point to obtain $f(x_1)$ and $f(x_2)$.

3. The Comparison-and-Shrink Step

At each iteration, the two function values are compared, and one side of the current bracket is discarded according to the following logic, which follows directly from the unimodality assumption:

1. If $f(x_1) > f(x_2)$: the function is higher at the left test point than at the right one. Under unimodality, the minimizer cannot lie to the left of x_1 , since the function would then have to decrease, reach a minimum, and increase twice over — contradicting unimodality. The left endpoint of the bracket is therefore advanced to $a \leftarrow x_1$.
2. If $f(x_1) \leq f(x_2)$: by the symmetric argument, the minimizer cannot lie to the right of x_2 , so the right endpoint is pulled in to $b \leftarrow x_2$.

In either case, exactly one of the two former test points becomes an endpoint of the new, narrower bracket, while the *other* former test point automatically becomes one of the two new interior points — its position, and consequently its already-computed function value, can simply be reused. A single new interior point is then generated using the *next* ratio of Fibonacci numbers in the sequence, and f is evaluated there only. This reuse is the key efficiency of the method: after the first iteration, *each subsequent step requires only one new function evaluation*, rather than two, because one of the two comparison points is always carried over from the previous step.

4. Why the Interval Shrinks at a Fibonacci Rate

Because the interior points are placed at Fibonacci-ratio fractions of the interval rather than at its midpoint, the width of the bracket after k iterations shrinks in proportion to F_{n-k} / F_n of the original width, rather than by a constant factor of one-half as in bisection. This ratio decreases *faster* than the constant halving rate for a given evaluation budget, which is what makes Fibonacci search provably optimal, among sequential search methods restricted to comparing pairs of function values, in the sense of minimizing the final interval width for a fixed number of function evaluations.

5. Termination and the Final Estimate

The comparison-and-shrink step is repeated for a total of $n - 2$ iterations, which is exactly the number determined at the outset from the Fibonacci-number generation criterion. By construction, this guarantees that the final bracket $[a, b]$ has width no larger than the specified tolerance. The minimizer is then reported as the *midpoint* of this final, very narrow bracket, $(a + b) / 2$, and the corresponding function value is reported as the estimated minimum. Throughout the process, the sequence of shrinking brackets can also be recorded, since it provides a natural way to visualize how quickly and monotonically the search converges toward the minimizer — the left endpoint rising and the right endpoint falling, iteration by iteration, until the two are indistinguishable to within the working tolerance.

6. Summary

In summary, Fibonacci search minimizes a unimodal function by (i) precomputing enough Fibonacci numbers to fix the number of iterations needed to reach a target tolerance, (ii) placing two interior test points according to Fibonacci ratios of the interval width, (iii) repeatedly discarding the sub-interval that unimodality guarantees cannot contain the minimizer, reusing one function evaluation from the previous step at each stage, and (iv) returning the midpoint of the final, tolerance-sized bracket as the minimizer once the fixed number of iterations is complete. Its principal advantage over simpler bracketing methods is evaluation efficiency: it achieves the guaranteed final interval width using the smallest possible number of function evaluations for a discrete, ratio-based search of this kind.